

SAPIENTIA ERDÉLYI MAGYAR TUDOMÁNYEGYETEM

Marosvásárhelyi Kar – Műszaki és Humántudományi Kar

Villamosmérnöki Tanszék

Szak: INFOKOMMUNIKÁCIÓS HÁLÓZATOK ÉS RENDSZEREK (TÁVKÖZLÉS)

ALKALMAZOTT INFORMATIKA ELMÉLET

Tantárgyfelelős / Laboratóriumi oktató:
Dr. Osztián Erika egyetemi adjunktus

Célcsoport:
Infokommunikációs hálózatok és rendszerek (Távközlés) szak, I. évfolyam

Tartalomjegyzék

1. 3. Labor: Verziókövetés Git és GitHub segítségével	1
1.1. A tananyag célja és relevanciája	1
1.2. 1. A verziókövetés elméleti alapjai és a Git architektúrája	2
1.2.1. Központi vs. Elosztott verziókövetés	2
1.2.2. A Git fájlok három belső állapota a saját számítógépen	2
1.3. 2. A folyamatos frissítés és verziókövetés elvei	2
1.3.1. Miért fontos a mérnöki munkában a ciklikus commit és push?	2
1.4. 3. A verziókövetési ciklus végrehajtása	3
1.4.1. A változat: GitHub Desktop grafikus felületen	3
1.4.2. B változat: Parancssorban / Terminalban (CLI)	3
1.5. 4. Gyakorlati feladatsor és önálló munka	5
2. 4. Labor: Haladó dokumentumszerkesztés MS Wordben és AI támogatás	6
2.1. A laboratórium célja és relevanciája	6
2.2. 1. Stílusok használata és automatikus tartalomjegyzék	7
2.2.1. Miért kulcsfontosságúak a stílusok?	7
2.2.2. A stílushierarchia beállítása	7
2.2.3. Automatikus tartalomjegyzék beszúrása	7
2.3. 2. Ábrák, táblázatok, hivatkozások és lábjegyzetek	7
2.3.1. Képaláírások (Captions) és Kereszthivatkozások	7
2.3.2. Lábjegyzetek (Footnotes)	8
2.4. 3. Műszaki dokumentumszerkesztés AI támogatással (AI + Office)	8
2.4.1. Mire használható felelősségteljesen az AI a MS Wordben?	8
2.5. 4. Gyakorlati feladatsor és önálló munka	9

1. fejezet

3. Labor: Verziókövetés Git és GitHub segítségével

1.1 A tananyag célja és relevanciája

A modern szoftverfejlesztésben, hálózاتمérnöki feladatoknál és műszaki projekteknél a **verziókövető rendszerek (VCS – Version Control System)** használata elengedhetetlen. A verziókövetés lehetővé teszi a forráskódok, dokumentációk és konfigurációs fájlok változásainak időbeli nyomon követését, a korábbi állapotok visszaállítását, valamint a hatékony csapatmunkát.

Az *Alkalmazott informatika* tantárgy követelményrendszerében a féléves gyakorlati osztályzat **70%-át a GitHub-on felépített és megvédett digitális portfólió adja**. Ebben a fejezetben elsajátítjuk a Git verziókövető rendszer elméleti alapjait, valamint felépítünk egy olyan munkafolyamatot, ahol a hallgatók a meglévő helyi repozitóriumukba (*Alkalmazott_informatika_2026*) egy önállóan megírandó, összetettebb C nyelvű mérnöki feladatot mentenek el, majd frissítik és módosítják azt a felhőbeli GitHub tárolójukban.

A fejezet fő tanulási célkitűzései

- A központi és az elosztott verziókövető rendszerek közötti különbségek megértése.
- A Git három belső állapotának (*Working Directory*, *Staging Area*, *Local Repository*) átlátása.
- A helyi tároló és a távoli (*remote*) GitHub felhőtárhely összekapcsolásának kezelése.
- A **GitHub Desktop** grafikus alkalmazás és a **Parancssor (CLI)** párhuzamos használata a verziókövetési ciklusban.
- Önálló mérnöki C program fájljainak verziókövetése, módosítása és a folyamatos feltöltés (*push*) gyakorlása.

1.2 1. A verziókövetés elméleti alapjai és a Git architektúrája

A hagyományos fájlkezelési módszerek (pl. `program_v1.c`, `program_v2_vegleges.c`) átláthatatlanok és hibaveszélyesek. A verziókövető rendszerek automatikusan elmentik a fájlok pillanatképét (*snapshot*), rögzítik a módosítás idejét, szerzőjét és a változtatások leírását.

1.2.1 Központi vs. Elosztott verziókövetés

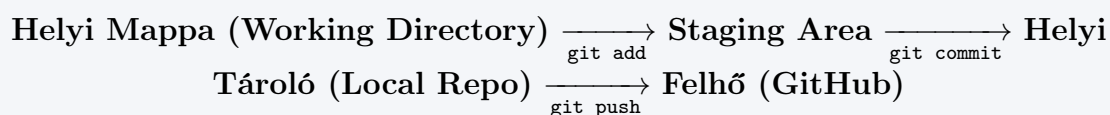
- **Központi verziókövetés (CVCS – pl. SVN):** Egyetlen központi szerver tárolja a teljes előzményt. Ha a szerver elérhetetlen vagy megsérül, a fejlesztők nem tudnak dolgozni vagy elvesz a projekt története.
- **Elosztott verziókövetés (DVCS – pl. Git):** Minden fejlesztő gépén megtalálható a teljes projekt és annak teljes verziótörténete. A helyi munkavégzéshez nincs szükség internetkapcsolatra, a távoli szerver (pl. GitHub) pedig elsősorban szinkronizációra és együttműködésre szolgál.

1.2.2 A Git fájlok három belső állapota a saját számítógépen

A Git a projekt fájljait három fő területen kezelheti a helyi számítógépen:

1. **Working Directory (Munkakönyvtár):** A számítógépünk fájlrendszerében (egy kijelölt helyi mappában) lévő tényleges fájlok, amelyeket a szerkesztőben (pl. VS Code, Overleaf) módosítunk.
2. **Staging Area / Index (Átmeneti tároló):** Egy előkészítő terület, ahová kijelöljük azokat a változtatásokat, amelyeket a következő mentésbe (*commit*-ba) be szeretnénk csomagolni.
3. **Local Repository (Helyi tároló / .git mappa):** A projekt véglegesen elmentett pillanatképeinek adatbázisa a saját számítógépen.

A Git alapvető adatáramlási modellje



1.3 2. A folyamatos frissítés és verziókövetés elvei

1.3.1 Miért fontos a mérnöki munkában a ciklikus commit és push?

Egy komplex szoftveres vagy távközlési feladat megoldásakor nem elegendő csak a végleges állapotot elmenteni. A jó mérnöki gyakorlat szerint minden egyes működő funkció vagy logika elkészülte után mentési pontot (*commit*-ot) kell létrehozni.

A ciklikus verziókövetési lépések:

1. **Modul elkészítése / módosítása:** A kód írása vagy javítása a helyi munkakönyvtárban.
2. **Staging** (`git add`): A módosított fájlok kijelölése elmentésre.
3. **Commit** (`git commit`): A pillanatkép rögzítése a helyi tárolóban egy egyértelmű üzenettel (pl. *"IPv4 maszk validáció megvalósítása"*).
4. **Push** (`git push`): A helyi mentések feltöltése a felhőbeli GitHub tárolóba.

1.4 3. A verziókövetési ciklus végrehajtása

A módosítások feltöltése kétféleképpen végezhető el: grafikus felületen a **GitHub Desktop** használatával, vagy közvetlenül **parancssorból (CLI)**.

1.4.1 A változat: GitHub Desktop grafikus felületen

1. Nyisd meg a **GitHub Desktop** alkalmazást, és válaszd ki a meglévő repozitóriumodat (`Alkalmazott_informatika_2026`)!
2. A bal oldali panelen automatikusan megjelennek a módosított vagy új fájlok (*Changes*).
3. Írj egy leíró üzenetet a bal alsó *Summary* mezőbe!
4. Kattints a **"Commit to main"** gombra!
5. Kattints a felső sávban található **"Push origin"** gombra a felhőbe való feltöltéshez!

1.4.2 B változat: Parancssorban / Terminalban (CLI)

1. Lépj be a helyi mappába a parancssorban:

```
$ cd C:/Informatika_Aplicata_Portfolio
```

2. Ellenőrizd a módosult fájlok állapotát:

```
$ git status
```

3. Add hozzá a módosított fájlokat az átmeneti tárolóhoz:

```
$ git add halozat_kalkulator.c
```

4. Mentsd el a helyi tárolóba leíró üzenettel:

```
$ git commit -m "Haloza ti kalkulator modositasa"
```

5. Töltsd fel a felhőbeli GitHub tárolóba:

```
$ git push origin main
```


1.5 4. Gyakorlati feladatsor és önálló munka

Gyakorlati Feladatsor – Távközlési C program felépítése és verziókövetése (Checklist)

A feladat során a hallgatóknak a meglévő meglévő repózitóriumukban (Alkalmazott_informatika_2026) önállóan kell megírniuk és két lépésben elmenteniük egy távközlési C programot.

Mérnöki C program feladat-specifikáció (Készítsd el önállóan!)

Feladat: Készíts egy C nyelvű `halozat_kalkulator.c` nevű programot az *Infokommunikációs hálózatok és rendszerek / Távközlés* szakos tanulmányaidhoz!

- **1. Verzió követelményei (Alapmodul):** A program kérje be a felhasználótól a bemeneti jel teljesítményét (P_{be}) és a kimeneti jel teljesítményét (P_{ki}) mW-ban (milliwatt). Számítsa ki és írja ki a csillapítást / erősítést decibelben (dB) a $A_{dB} = 10 \cdot \log_{10}(P_{ki}/P_{be})$ képlet alapján!
- **2. Verzió követelményei (Kiterjesztett modul):** Bővítsd ki a programot egy új funkcióval, amely átváltja a bemeneti P_{be} teljesítményt dBm értékre ($P_{dBm} = 10 \cdot \log_{10}(P_{be})$), valamint ellenőrzi, hogy a bemeneti teljesítmény pozitív-e ($P_{be} > 0$).

A változat: Munkamenet GitHub Desktop felületen (GUI)

- [] **1. feladat (Alapmodul megírása):** Nyisd meg a helyi repózitóriumod mappáját (Alkalmazott_informatika_2026), és hozd létre a `halozat_kalkulator.c` fájlt! Írd meg az 1. verzió követelményeinek megfelelő C kódot!
- [] **2. feladat (1. Commit és Push):** Nyisd meg a **GitHub Desktop** alkalmazást! A bal oldali sávban észleli az új `halozat_kalkulator.c` fájlt. Írd be a *Summary* mezőbe: "1. verzio: dB csillapitas kalkulator megirasa", kattints a "**Commit to main**" gombra, majd nyomd meg a felső "**Push origin**" gombot!
- [] **3. feladat (Kód kiterjesztése):** Nyisd meg újra a `halozat_kalkulator.c` fájlt a szerkesztődben (pl. VS Code), és valósítsd meg a 2. verzió követelményeit (dBm átváltás és bemenet-ellenőrzés)!
- [] **4. feladat (2. Commit és Push):** Váltss vissza a **GitHub Desktop**-ra! A program jelzi a kódmódosítást. Írd be a *summary*-be: "2. verzio: dBm atvaltas es hibakezeles hozzaadasa", kattints a "**Commit to main**" gombra, majd nyomd meg a "**Push origin**" gombot!
- [] **5. feladat (Ellenőrzés):** Nyisd meg a GitHub repózitóriumodat a böngészőben (*Repository* → *View on GitHub*), és ellenőrizd a `halozat_kalkulator.c` frissült tartalmát, valamint a történetben lévő 2 külön commit-ot!

B változat: Munkamenet Parancssorban / Terminalban (CLI)

- [] **1. feladat (Alapmodul megírása):** Lépj be a helyi repózitórium mappájába, és hozd létre a `halozat_kalkulator.c` fájlt az 1. verzió specifikációja szerint:

```
$ cd C:/Informatika_Aplicata_Portfolio
```

2. fejezet

4. Labor: Haladó dokumentumszerkesztés MS Wordben és AI támogatás

2.1 A laboratórium célja és relevanciája

A távközlési és infokommunikációs mérnöki gyakorlatban a műszaki dokumentációk, mérési jegyzőkönyvek, projektjelentések és a diplomamunkák (szakdolgozatok) elkészítése során elengedhetetlen a professzionális dokumentumszerkesztési ismeret.

Míg a tudományos publikációk jelentős része LaTeX környezetben készül, a vállalati és ipari szférában a Microsoft Word az elsődleges dokumentumszerkesztő szoftver. A közvetlen, kézi formázás (szóközökkel vagy Enterekkel való igazítás, egyedi betűméretek manuális átállítása) átláthatatlan, nehezen módosítható és nem felel meg a mérnöki követelményeknek.

Ebben a laboratóriumi gyakorlatban elsajátítjuk a címsorstílusok használatát, az automatikus tartalomjegyzék és képaláírások generálását, valamint a Mesterséges Intelligencia (Microsoft Copilot, ChatGPT) felelősségteljes integrálását a szövegek strukturálására és nyelvi tisztázására.

A fejezet fő tanulási célkitűzései

- A stílusos dokumentumszerkesztés elveinek és a stílushierarchiának (*Címsor 1, Címsor 2, Címsor 3*) elsajátítása.
- Automatikus tartalomjegyzék (*Table of Contents*) generálása és frissítése.
- Ábrák, táblázatok, képaláírások (*Captions*) és kereszthivatkozások (*Cross-references*) beszúrása és indexelése.
- AI asszisztensek (*Microsoft Copilot*) etikus használata szövegtömörítésre, átfogalmazásra és hibalistázásra.
- Az elkészített dokumentum beillesztése a helyi Git tárolóba és feltöltése (*push*) a GitHub portfólióba.

2.2 1. Stílusok használata és automatikus tartalomjegyzék

2.2.1 Miért kulcsfontosságúak a stílusok?

A stílusok (*Styles*) előre definiált formázási szabálygyűjtemények (betűtípus, betűméret, térköz, sorköz, bekezdésigazítás). Használatuk előnyei:

- **Egységesség:** Az egész dokumentumban garantáltan azonos megjelenésűek lesznek a fejezetcímek és a törzsszövegek.
- **Gyors módosítás:** Ha megváltoztatjuk a *Címsor 1* stílus tulajdonságait, a dokumentumban található összes I. szintű fejezetcím automatikusan frissül.
- **Automatizálás:** Ezen stílusok alapján generálható a navigációs ablak és az automatikus tartalomjegyzék.

2.2.2 A stílushierarchia beállítása

1. **Címsor 1 (Heading 1):** Főfejezetek címei (pl. *1. Bevezetés*).
2. **Címsor 2 (Heading 2):** Alfejezetek címei (pl. *1.1. A hálózat architektúrája*).
3. **Címsor 3 (Heading 3):** Részletes alfejezetek (pl. *1.1.1. A router konfigurációja*).
4. **Normál (Normal):** A dokumentum törzsszövege (sorkizárt igazítás, 1.15–1.5 sorköz, 6pt bekezdés utáni térköz).

2.2.3 Automatikus tartalomjegyzék beszúrása

Miután a fejezetcímeket elláttuk a megfelelő Címsor stílusokkal, a tartalomjegyzék beszúrása:

- Válasszuk a **Hivatkozás (References) → Tartalomjegyzék (Table of Contents)** menüpontot!
- A dokumentum módosítása után a tartalomjegyzékre jobb gombbal kattintva a **Megfrissítés (Update Field) → A teljes jegyzék frissítése** opcióval frissíthetők az oldalszámok és fejezetcímek.

2.3 2. Ábrák, táblázatok, hivatkozások és lábjegyzetek

Műszaki dokumentumokban tilos az ábrákat és táblázatokat kézzel számozni a szövegben.

2.3.1 Képaláírások (Captions) és Kereszthivatkozások

1. **Ábra / Táblázat beszúrása:** Illesszük be az ábrát (középre igazítva).
2. **Képaláírás hozzáadása:** Jobb gomb az ábrára/táblázatra → **Képaláírás beszúrása (Insert Caption)**.
 - Címke: *1. ábra:* vagy *1. táblázat:*.

- Megnevezés: 1. ábra: A router belső szerkezete.

3. **Kereszthivatkozás (Cross-reference):** A törzsszövegben való hivatkozáshoz használjuk a **Hivatkozás** → **Kereszthivatkozás** opciót (pl. "az 1. ábrán látható módon..."). Így a képek átrendezésekor a hivatkozott számozás automatikusan frissül!

2.3.2 Lábjegyzetek (Footnotes)

Rövid magyarázatok, szakmai kifejezések vagy weboldal hivatkozások megadására szolgál:

- Kattintsunk a kifejezés mögé, majd választuk a **Hivatkozás** → **Lábjegyzet beszúrása (Insert Footnote)** opciót!

2.4 3. Műszaki dokumentumszerkesztés AI támogatással (AI + Office)

A modern dokumentumszerkesztést jelentősen megkönnyítik a generatív AI asszisztensek (pl. Microsoft Copilot, ChatGPT). Az AI felelősségteljes használata növeli a hatékonyságot, de nem helyettesíti a mérnöki ellenőrzést.

2.4.1 Mire használható felelősségteljesen az AI a MS Wordben?

- **Szövegek strukturálása:** Hosszú, tömör jegyzetek vázlattá vagy pontokba szedett listává alakítása.
- **Nyelvi és stilisztikai tisztázás:** Helyesírás-ellenőrzés, körmondatok átfogalmazása és műszaki stílusra igazítás.
- **Összefoglalók készítése:** Hosszabb szakmai fejezetekből 3–4 mondatos absztrakt vagy vezetői összefoglaló generálása.

AI Prompt Sablonok MS Word Dokumentumokhoz

1. **Prompt – Műszaki stílusra igazítás és tördelés:** "Olvasd el az alábbi műszaki szövegrészletet! Javítsd ki a helyesírási hibákat, formázd át átlátható bekezdésekre, és fogalmazd át hivatalos, mérnöki stílusban, megtartva az szakszavakat:"

2. **Prompt – Strukturált vázlat készítése:** "Készíts egy hierarchikus vázlatot (Címsor 1, Címsor 2 szinten) egy 'Infokommunikációs hálózatok biztonsága' című műszaki jelentéshez!"

2.5 4. Gyakorlati feladatsor és önálló munka

4. Labor Gyakorlati Feladatsor (Checklist)

Végezd el az alábbi lépéseket az MS Word szoftverben és a helyi Git tárolóban:

- [] **1. feladat:** Nyiss egy új MS Word dokumentumot `Muszaki_Jelentes_Nev.docx` néven!
- [] **2. feladat:** Hozz létre egy címlapot a dokumentum elején (Cím: *Infokommunikációs Hálózatok*, Alcím: *Műszaki Jelentés*, Szerző: *A te neved*, Szak: *Infokommunikációs hálózatok és rendszerek / Távközlés*)!
- [] **3. feladat:** Készíts legalább 3 főfejezetet (*Címsor 1*) és 2 alfejezetet (*Címsor 2*)!
- [] **4. feladat:** Állítsd be a **Normál** stílust: Times New Roman vagy Calibri, 12pt, sorkizárt, 1.15-ös sorköz!
- [] **5. feladat:** Szúrj be egy automatikus tartalomjegyzéket a címlap utáni második oldalra!
- [] **6. feladat:** Illessz be egy hálózati ábrát vagy táblázatot, adj hozzá hivatalos képaláírást (*Caption*), és hivatkozz rá a szövegben kereszthivatkozással (*Cross-reference*)!
- [] **7. feladat:** Használd AI asszisztenst (Copilot / ChatGPT) az egyik fejezet szövegének átfogalmazására és nyelvi javítására!
- [] **8. feladat:** Mentsd el a kész dokumentumot PDF formátumban is (`Muszaki_Jelentes_Nev.pdf`)!
- [] **9. feladat:** Másold át a PDF fájlt a helyi Git portfólió mappádba (`Informatica_Aplicata_Portfolio`)!
- [] **10. feladat:** Hajtsd végre a feltöltési ciklust GitHub Desktop-ban vagy parancssorból: `git add .`, `git commit -m "4. labor MS Word jelentés hozzáadása"`, majd `git push`!